

Usuarios y seguridad

El soporte para múltiples usuarios en un sistema requiere que el SO deba definir e implementar los siguientes mecanismos:

1. *Separación de ambientes de trabajo* para cada usuario en el sistema de archivos.
2. *Identificar y autenticar* usuarios.
3. Relacionar programas, procesos y usuarios.
4. Establecer límites en la utilización de recursos del sistema para permitir que todos los usuarios logren sus objetivos.
5. Ofrecer servicios (comandos y syscalls) de gestión de usuarios.
6. Ofrecer mecanismos de *control de acceso* a recursos.
7. Opcionalmente, ofrecer múltiples terminales/consolas (reales o virtuales).

Usuarios, grupos y archivos

Un SO comúnmente asocia a un usuario con un identificador único (ej: en UNIX es un entero positivo), un rótulo (*nombre de usuario*) y credenciales de autenticación.

Un *grupo* representa un conjunto de usuarios. Comúnmente se usan para facilitar la gestión de control de acceso.

Un *recurso* es un archivo de datos o programa, mecanismo de IPC (ej: un pipe), proceso o dispositivo de hardware del sistema.

En todo sistema hay al menos un *usuario privilegiado* que es quien podrá crear nuevos usuarios y asignarles roles o permisos. Este usuario no tiene restricciones de acceso a los recursos y comúnmente actúa como el administrador del sistema. En sistemas tipo UNIX éste usuario se denomina *root*, mientras en MS-Windows es *Administrador*.

El sistema ofrece utilidades para gestionar usuarios y grupos como `add/user` y `add/group`.

En los sistemas tipo UNIX las bases de datos de usuarios se encuentran en los archivos `/etc/passwd` y `/etc/shadow`. El primero contiene tuplas o registros de la forma `user:x:uid:gid:full name:homedir:shell`. Comúnmente una configuración de un sistema incluye usuarios que no tienen login (usan el programa `/usr/bin/nologin` como shell), como `bin`, `sys`, `mail`, etc. Estos usuarios permiten asociar el *dueño*, *grupo* y *permisos* a programas o servicios del sistema.

El segundo campo, si contiene una `x` significa que su contraseña está en el archivo `/etc/shadow`. Es interesante notar los permisos asociados a esos archivos.

En estos sistemas el usuario *root* tiene identificador 0 (*uid:0*). Comúnmente al primer usuario real del sistema se le asigna el *uid=1000*.

Cada usuario tiene su *grupo principal* y puede ser miembro de otros grupos. La base de datos está en el archivo `/etc/group`.

Login

El sistema ejecuta en cada terminal el programa `login`, el cual generalmente solicita el nombre de usuario y credenciales de autenticación. El método más común es la presentación de una *contraseña* o *password*. Cada usuario puede cambiar su contraseña con el comando `passwd` (en sistemas tipo UNIX). El administrador comúnmente asigna la contraseña inicial durante la creación del usuario.

Las contraseñas se almacenan en el archivo `etc/shadow` cuyo formato es el siguiente:

```
user:hashed-password:last-changed:min-days:max-days:warn-days:inactive
```

donde el primer campo es el nombre del usuario, en el segundo está el *hash del password*, comúnmente prefijado por `$n$` el cual indica el código del algoritmo de hash usado. Los demás campos contienen fechas o días desde el último cambio, período en que el sistema debería requerir el cambio de contraseña, los días de inactividad y la fecha de expiración de la cuenta.

Para no almacenar secretos en forma plana, se almacena el *hash(password)*. La función *hash* es un *hash criptográfico*, como MD5 o SHA. Estas funciones son de la forma $hash : \{0, 1\}^* \rightarrow \{0, 1\}^n$ donde n es el número de bits de la salida. Es decir que toman una entrada de longitud arbitraria y generan una salida de longitud fija. Por ejemplo, *md5* genera una salida de 128 bits.

Note

Una de las propiedades deseables de una función de hash criptográfico es que dados el par $(x, hash(x))$ es muy difícil (requiere un gran poder de cómputo por ser un problema NP-Hard) encontrar x' tal que $hash(x') = hash(x)$. Actualmente es relativamente fácil romper esta propiedad para md5, por lo que se considera *quebrado* y no se aconseja su uso para este fin.

El programa `login` autentica al usuario verificando que $hash(provided_password) = saved_password$.

Comúnmente el comando `passwd` agrega al password provisto por el usuario un prefijo o *salt*: Un valor generador aleatoriamente (en UNIX dos caracteres) y lo almacena como prefijo del hash. Esto previene *ataques por diccionario*.

Note

Este ataque puede aplicarse ante un *robo* del archivo de contraseñas por el atacante. El atacante dispone (generado previamente) un *diccionario*: conjunto de pares (*plaintext*, *hash(plaintext)*). Si una contraseña (hash) coincide con el segundo elemento de algún par, ya conoce el texto plano correspondiente. El problema de inferir la contraseña se resuelve mediante una búsqueda. El *salt* requiere que la generación de *buenos* diccionarios sea muchísimo mas complejo.

El sistema de login y autenticación puede realizar presentando otras credenciales o usando otros mecanismos. Por ejemplo, los sistemas físicos pueden usar huellas dactilares, reconocimiento de rostro u otros mecanismos.

También pueden usarse múltiples factores. Por ejemplo la de doble factor se basa en que adicionalmente el usuario debe proveer un código enviado por un servicio de mensajería (mail, chat, etc) o usando generador de códigos de autenticación.

El SO GNU/Linux ofrece un sistema de autenticación muy flexible, conocido como *pluggable authentication modules (PAM)*. Este sistema permite configurar usar y desarrollar mecanismos que pueden combinarse. Para más información ver [PAM](#).

Control de acceso

El sistema deberá asociar permisos de acceso (ej: *lectura, escritura o ejecución*) de los usuarios a los recursos (archivos de datos y programas).

Esto puede hacerse mediante una *matriz de control de acceso*.

resources/users	f1	f2	...
root	rwX	rw-	...
...	...	...	...
alice	r--	---	...

Implementar tal matriz es un desafío ya que podría ser inmensa por la cantidad de recursos (y usuarios).

Listas de control de acceso (ACLs)

Es una implementación por columnas. A cada recurso se le asocia el conjunto de usuarios y permisos de acceso: $\{(user, perms), \dots\}$.

Para lograr una representación compacta de este conjunto, los usuarios se clasifican en tres clases:

1. *U (user)*: El *dueño (owner)* del archivo.
2. *G (group)*: El *grupo (principal)*.
3. *O (others)*: El resto de los usuarios del sistema.

Los *permisos* se asignan al recurso para cada clase: Permisos de acceso del usuario *dueño* del recurso, permisos de acceso a los usuarios del grupo asociado y permisos para el resto de los usuarios.

Esto permite una representación muy compacta de una ACL: Sólo son necesarios 9 bits: Los tres primeros (lectura, escritura, ejecución o *rwx*) aplican al *dueño*, los tres siguientes al *grupo* y los tres últimos al resto.

$$\underbrace{rwx}_{U} \underbrace{r--}_{G} \underbrace{r--}_{O}$$

Estos permisos forman parte de los *metadatos* de cada archivo (en cada *inode*).

En los sistemas tipo UNIX, los comandos `chown` y `chgrp` permiten asignar el dueño y el grupo a archivos. El comando `chmod` permite asignar permisos.

Se debe notar que la agrupación de permisos en tres clases de usuarios a veces no provee la flexibilidad requerida. Los SO modernos ofrecen *full-acls*, que permiten asociar el conjunto de pares mencionados de manera explícita. Se implementan comúnmente como archivos especiales (ocultos) asociados a cada archivo. En GNU-Linux, ver los comandos `getfacl` y `setfacl`.

Un sistema que permite a los usuarios modificar los permisos de sus recursos (de los cuales son *owners*) se denomina *discretionary access control* (DAC).

Existen modelos de seguridad en que los permisos de acceso son asignados por una autoridad de seguridad. Por ejemplo, el modelo Bell-LaPadula asigna a los usuarios un *nivel de seguridad* y a los recursos un *rótulo*.

Este modelo comúnmente se usa en organizaciones militares y se conocen como modelos de *mandatory access control* (MAC).

En éstos modelos existe un orden parcial (retículo) de niveles de seguridad. Por ejemplo *TOP SECRET* > *SECRET* > *CONFIDENTIAL* > *PUBLIC*.

Por ejemplo, un *general* puede tener un nivel de seguridad *SECRET* lo que le permitirá acceder a recursos {*SECRET*, *CONFIDENTIAL*, *PUBLIC*} pero no podrá acceder a recursos *TOP SECRET*.

Privilegios de un proceso

Un proceso usará los permisos de su *usuario real*: El usuario que lanzó su ejecución. Esto hará que el proceso pueda acceder a recursos permitidos a ese usuario.

El *effective UID* se corresponde al usuario *dueño* del programa. En casos que un programa tenga activado el permiso *setuid*, al ejecutarse, el *uid* del proceso será el dueño del programa.

Por ejemplo, el programa `/usr/bin/passwd` (con dueño=*root*) requiere acceso a `/etc/shadow` el cual sólo tiene permisos de lectura y escritura sólo para *root* (para evitar su copia por parte de cualquier usuario). Pero este programa puede ser ejecutado por cualquier usuario para cambiar su contraseña. Por eso, este programa tiene activado el bit *setuid* para que cuando sea ejecutado adquiera los permisos de *root*.

Este es un mecanismo de *escalado de privilegios*. Otros mecanismos de cambios de privilegio los proveen los comandos `su` y `sudo`.

El primero permite crear una nueva sesión (*shell*) con otro usuario (por omisión *root*). El segundo permite ejecutar un comando como otro usuario.

El comando `su` solicita las credenciales de autenticación del otro usuario, el cual debe tener permitido el *login*.

El comando `sudo` sólo requiere que se re-ingresen las credenciales del usuario actual.

La configuración de `su` se encuentra en el archivo `/etc/sudoers` y especifica qué usuarios pueden ejecutar qué comandos bajo los permisos de otros usuarios.

En MS-Windows el comando equivalente a `su` es `runas`.

Capacidades

Muchas veces que un proceso adquiriera privilegios de *root* (todos) podría generar problemas de seguridad ya que requiere que el programa no contenga errores y que acceda sólo a los recursos necesarios.

Las capacidades permiten definir qué operaciones se pueden aplicar sobre recursos. Comúnmente se usan para *limitar* el conjunto de operaciones posibles. Por ejemplo, si se abre un archivo en modo de sólo lectura, el descriptor contiene sólo la capacidad de lectura, inhibiendo una operación de escritura o truncado.

Muchos SO modernos usan capacidades en programas privilegiados para limitar el conjunto de operaciones o recursos a los que pueden acceder.

Por ejemplo, el programa `tcpdump` permite *capturar* paquetes que pasan por una interface de red. Para ésto requiere privilegios de *root*. Comúnmente se puede aplicar una capacidad para que sólo se le permita acceder a *sockets raw* (`CAP_NET_RAW` en GNU/Linux).

Para más información de capacidades en GNU/Linux, hacer `man capabilities`.

[< Anterior](#)

[Próximo >](#)

Sistemas de archivos

Taller xv6